PlugPlan SG: Class Diagrams

SC2006 · Lab 5 Final · Author: Team SCE3-04 · Prepared with AI assistance (Claude Code) · 10 November 2026

Reading these diagrams

These diagrams describe the current code in **PlugPlan SG/src** and show its key classes. Most of the codebase is TypeScript modules, React function components and interfaces. A module or component is drawn as a class, its operations are its real functions, and the stereotypes give its analysis role. Only `ApiError` (client and server, drawn as ClientApiError and ServerApiError) and `ProviderError` are `class` declarations in the code.

+ means exported from the module, or a UI event handler. - means private to the module, or component state. Classes without members in a figure are context; their details are in the figure where they are primary. Small presentational components and the health route are omitted.

Key classes

Class	Role	Implementation	Responsibility
PlugPlanApp	Boundary	`components/plugplan-app.tsx`	Driver and Moderator screens and their state
RouteHandler	Boundary	`app/api/app/route.ts`	Single authenticated API entry point
ProviderGateway	Boundary	`lib/server/providers.ts`	LTA DataMall and OneMap access
Repositories	Boundary	`lib/server/db.ts`	Supabase persistence
ServiceHandler	Control	`lib/server/service.ts`	Coordinates each application action
PlanEngine	Control	`lib/domain/engine.ts`	Feasibility, charging estimates and ranking
AuthGuard	Control	`lib/server/supabase.ts`	Authentication and Driver/Moderator gates
PlanResult, IssueReport and related types	Entity	`lib/types.ts`	Domain data

Design patterns and extensibility

Design element	Where it appears	Benefit
Layered architecture	UI, one API route, ServiceHandler, PlanEngine, repositories, providers	Each layer can change without rewriting the others.
Facade / controller	`handleAction` behind `POST /api/app`	One entry point applies authentication, validation and error mapping.
Strategy	`Strategy`, `assignScores`, `compareCandidates`	A new ranking preset is a local change.
Adapter / gateway	`ProviderGateway`	A provider can be replaced without touching PlanEngine.
Repository	Functions in `db.ts`	Persistence stays behind named operations with row-level security.
Pure domain core	`calculatePlan` has no I/O	Calculation is deterministic and testable with fixtures.

Figure 1. System overview

Layered dependencies between the main boundary, control and persistence elements. Members are shown in the detailed views.

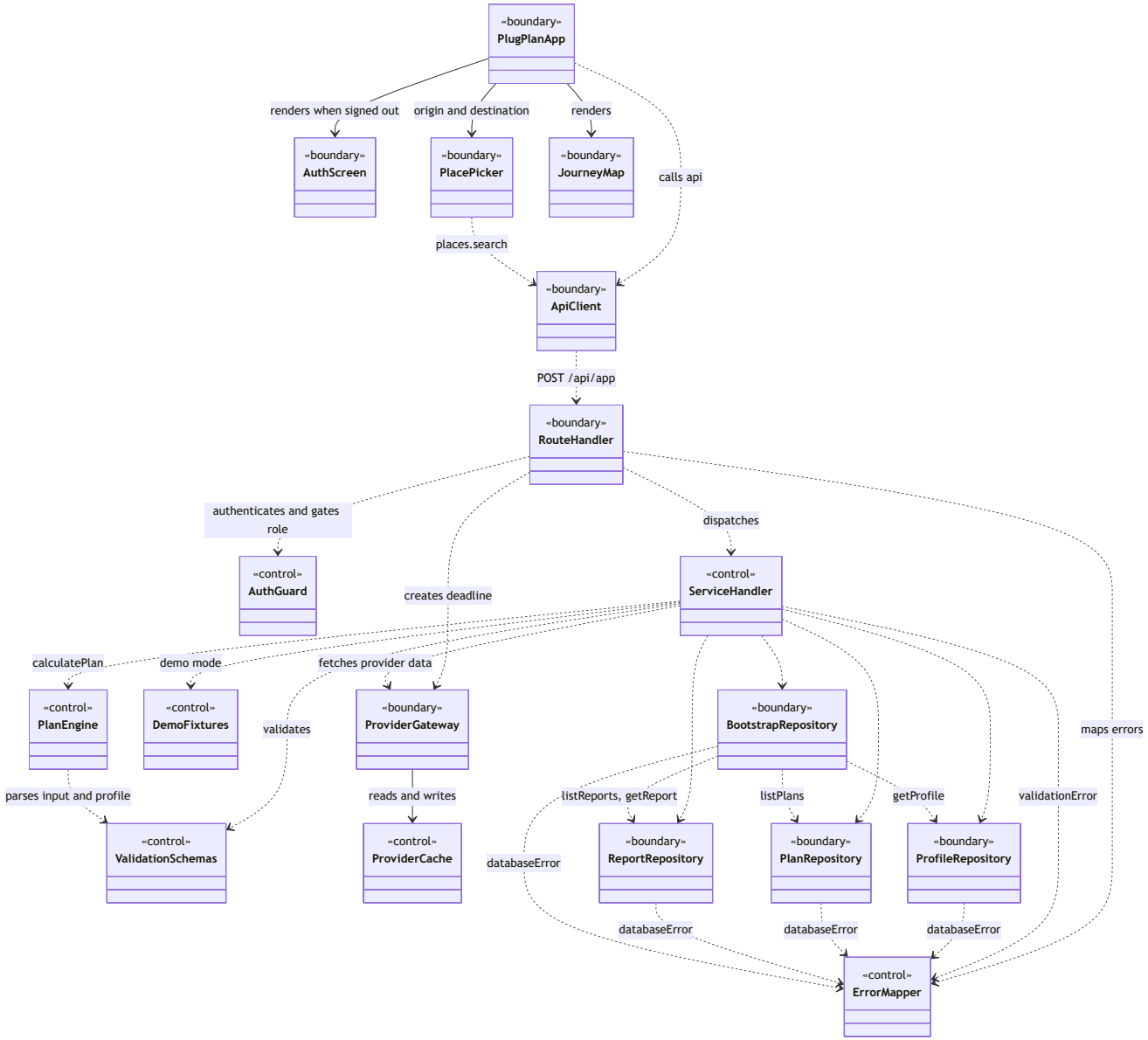


Figure 2. User interface and request boundary

PlugPlanApp owns the screen state. All application calls go through ApiClient to the single POST /api/app route. New since Lab 3: MapUtilities.googleMapsDirectionsUrl.

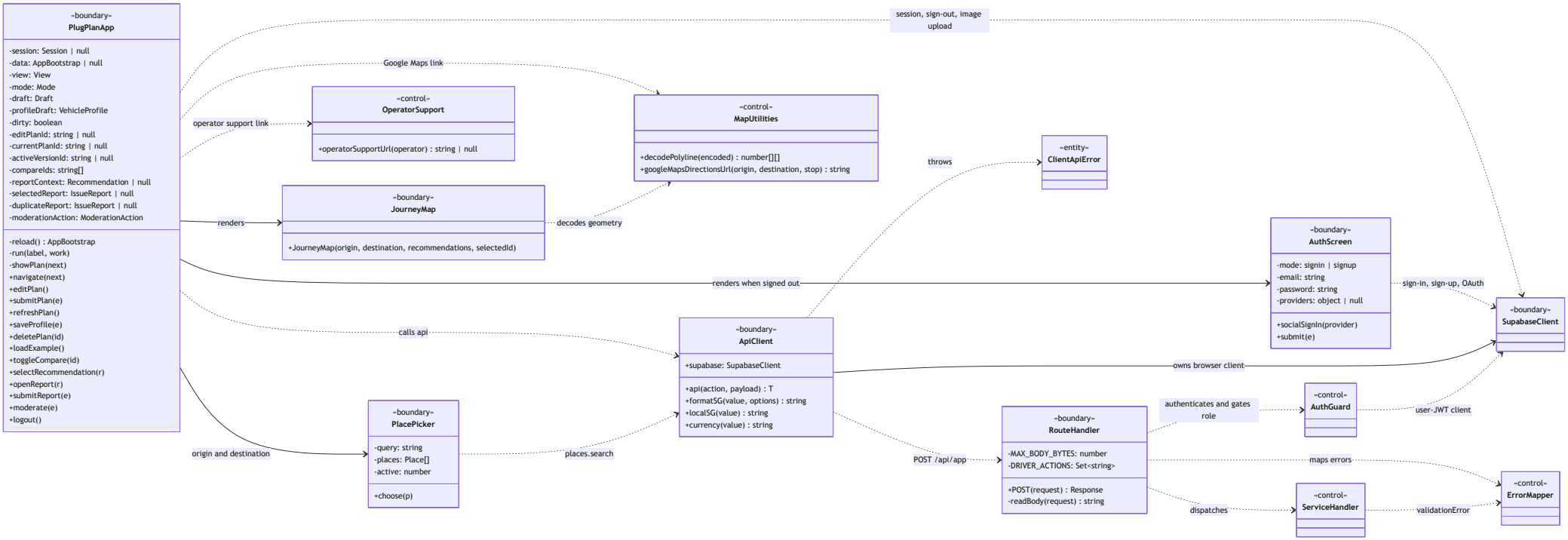


Figure 3. Planning control (UC-03, UC-05, UC-06)

ServiceHandler coordinates data retrieval and persistence; PlanEngine is the pure calculation. Strategy-based ranking lives in assignScores and compareCandidates.

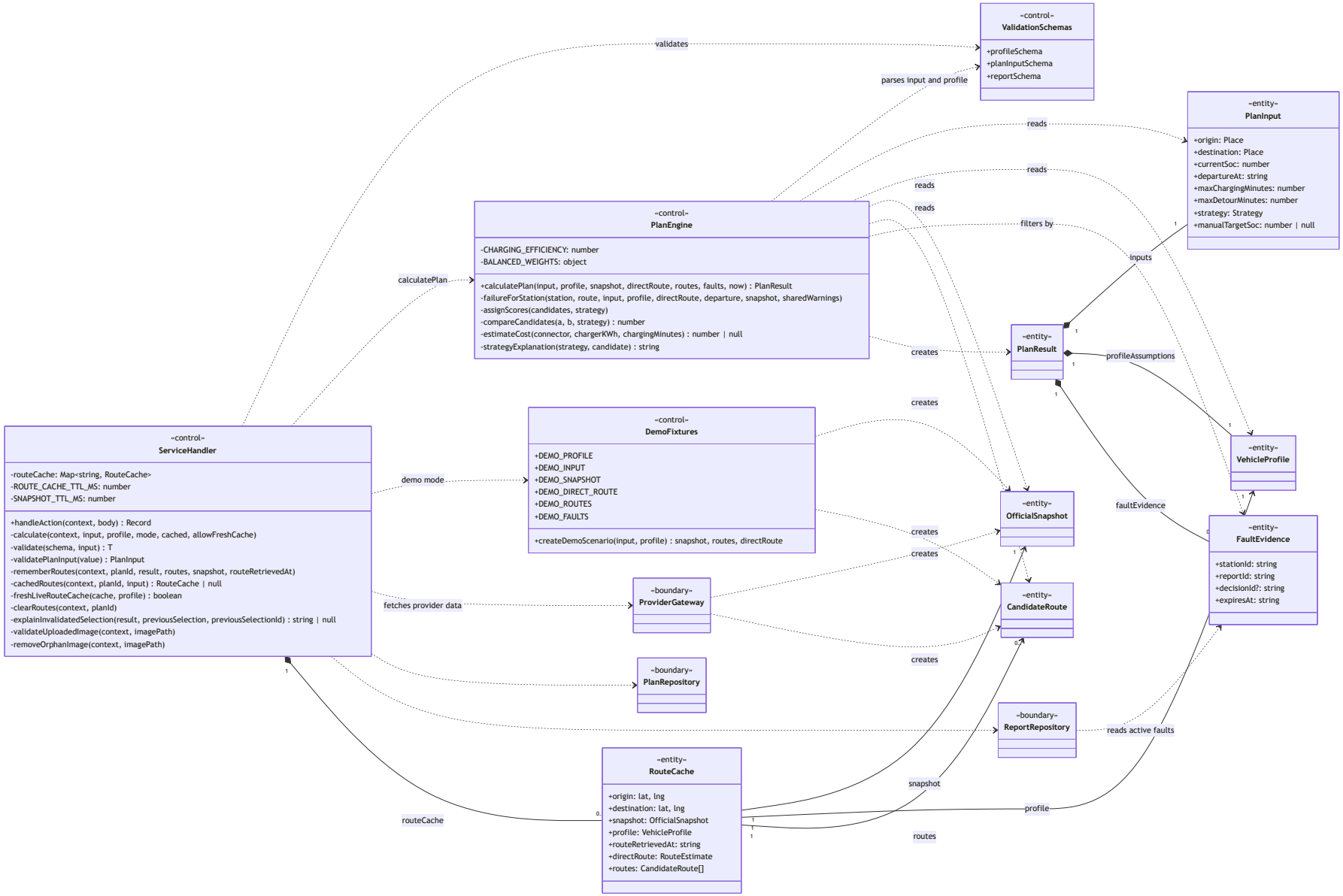


Figure 4. External data providers and caching

ProviderGateway adapts LTA DataMall and OneMap. Every external call in a request shares one Deadline; the cache supports labelled degraded results.

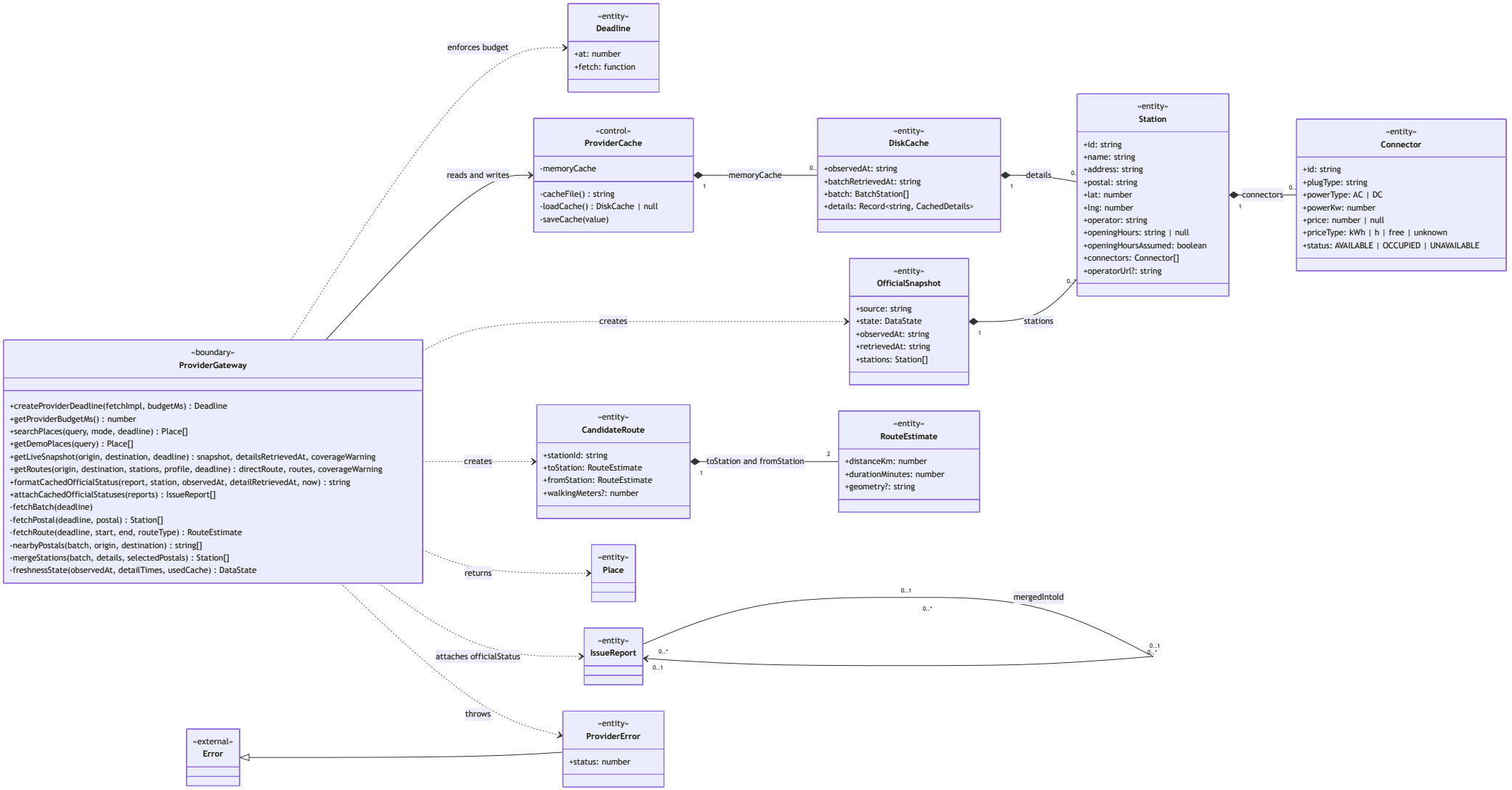


Figure 5. Security, persistence and errors

requireUser creates a RequestContext holding a user-JWT database client, so row-level security stays in force. The repositories group the functions of db.ts.

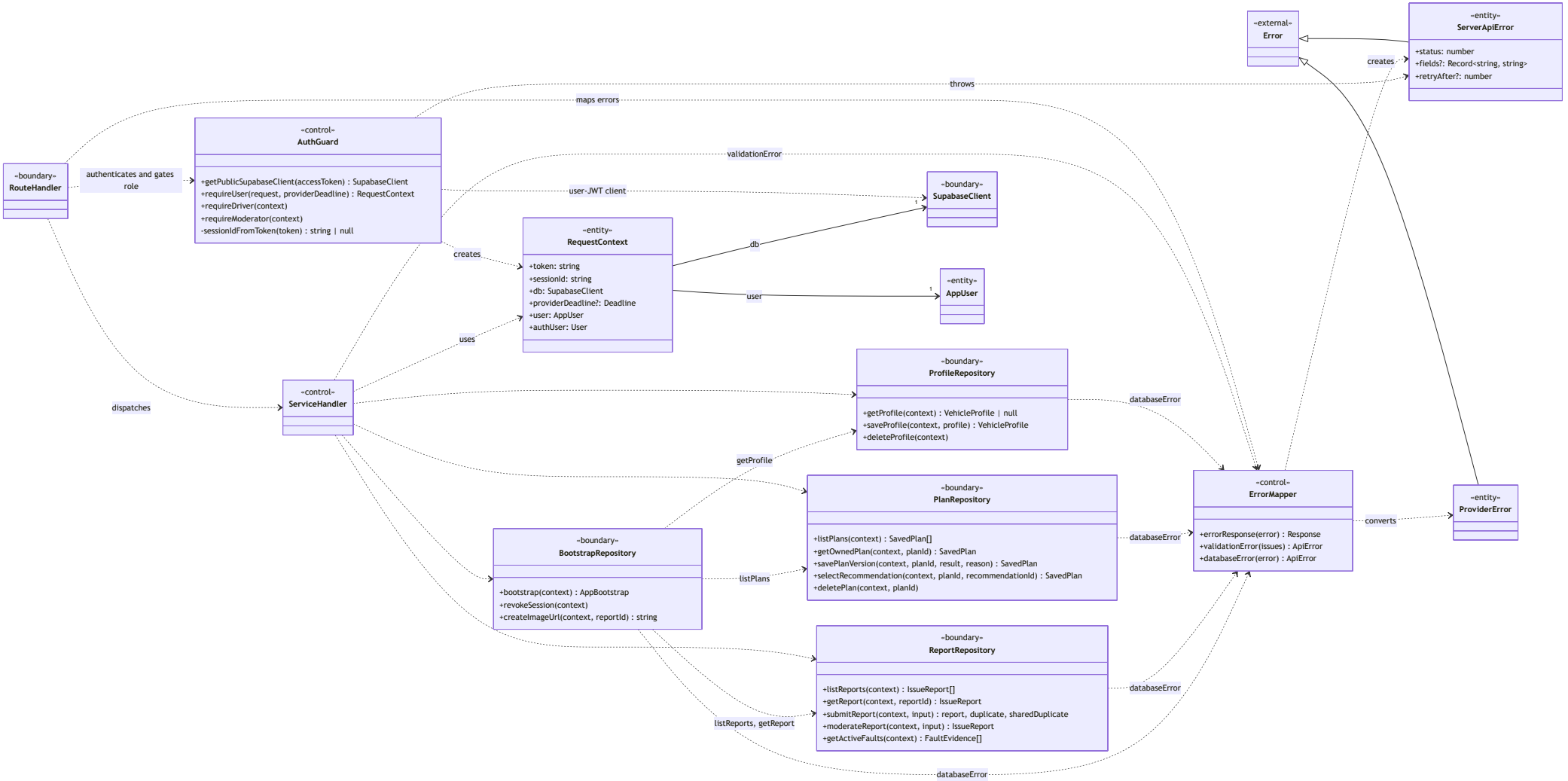


Figure 6. Plan domain model

A SavedPlan owns immutable PlanVersions; each version stores a complete PlanResult, including the inputs and vehicle assumptions used.

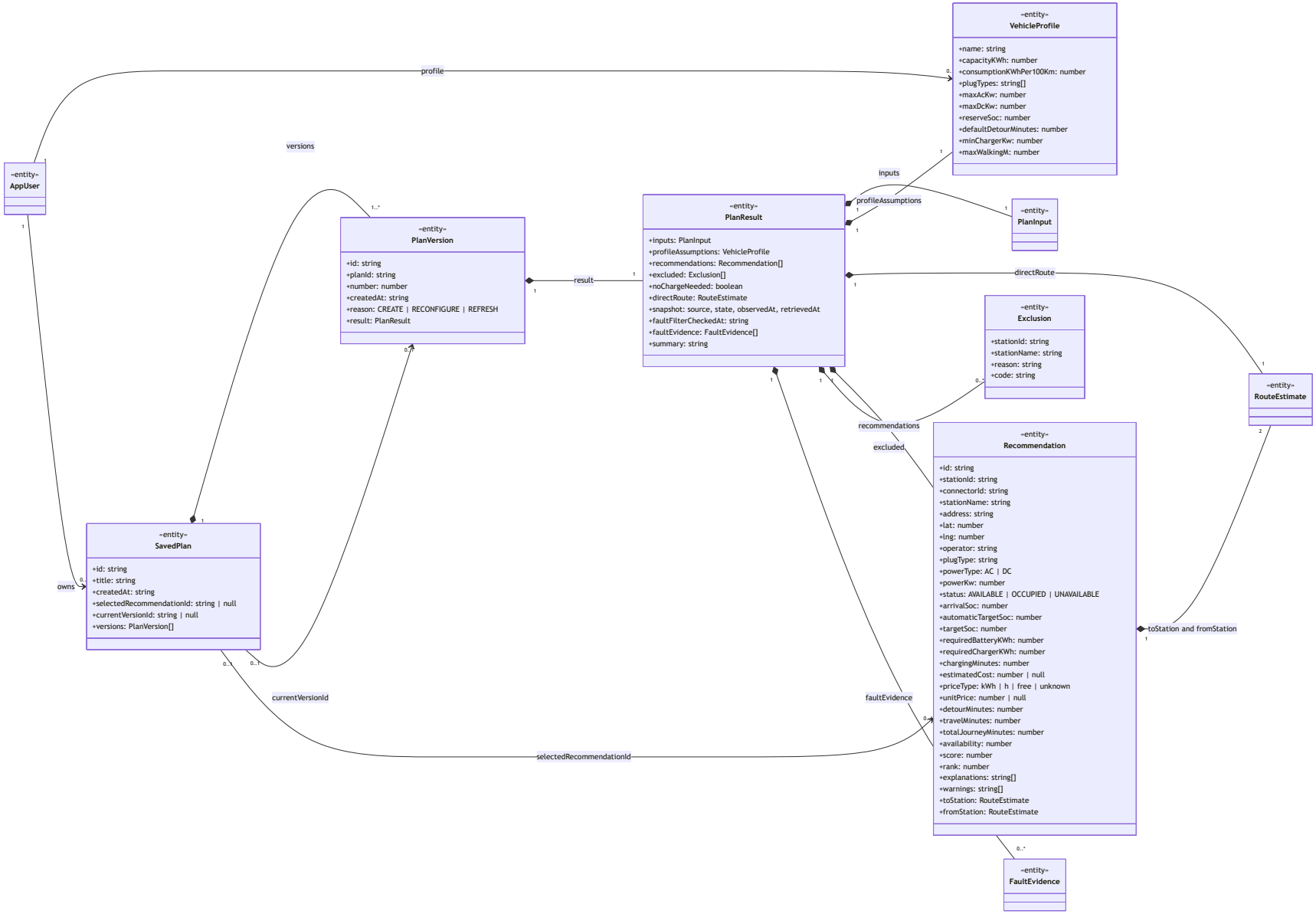


Figure 7. Accounts and community reports (UC-01, UC-07, UC-08)

A report keeps a revision for optimistic concurrency. Each Moderator decision is appended to its audit history. Only verified, unexpired fault reports become FaultEvidence.

